Bawabu demo: recording and voiceover script

Your rehearsal script for the 3-minute video, built from `demo-video-guide.md`. It has three parts, done in this order:

1. **Part 1, the screen recording:** what to record, where, every command to type, and what you should see. No talking while recording.
2. **Part 2, the voiceover:** exactly what to say, line by line, recorded afterwards while watching the footage.
3. **Part 3, putting it together:** which card, clip and line goes at each second of the video.

What you'll have at the end

	What	Count
Title cards	Already made, in `title cards/`	9 images
Take A	The live demo: Muse Code on the left, the Bawabu live report on the right	1 main take + 1 backup
B-roll	Short clips: B1, B3–B5, B7, B9 (plus spares B2, B6, B8)	4 needed + 3 spares
Voiceover	V1 to V8, one file per segment	8 audio files

Theme: this script uses **dark** everywhere (dark cards, dark report page, dark terminal). If you choose light instead, use the cards in `title cards/light mode/`, set the report page to light, and use a light terminal theme. Never mix the two in one video.

Part 1 · The screen recording

1.1 Set up once (the day before, about 30 minutes)

Step 1: Quiet the Mac

1. Turn on **Do Not Disturb**: Control Centre (top right of the menu bar) → **Focus** → **Do Not Disturb**.
2. Hide the Dock: **System Settings** → **Desktop & Dock** → **"Automatically hide and show the Dock"** → on.
3. Quit everything that can pop up: Slack, Mail, Messages, WhatsApp, calendar, music.
4. Use a plain dark desktop wallpaper, and move files off the desktop.

Step 2: The terminal

1. Open **Terminal** (or iTerm). Pick a dark profile.
2. Set the font to **20 pt**: Terminal → Settings → Profiles → Text → Font → Change… → size 20.
3. Make the prompt short, so your name and folders don't show. Type:

```
export PS1='$ '
```

Step 3: The browser (Chrome)

1. Open a **new window** just for the demo: File → New Window. Close every other tab in it.
2. Hide the bookmarks bar: **Cmd + Shift + B**.
3. Open the live page:

```
https://amohntim123--bawabu.us-east.modal.run/live
```

4. Zoom to **125%**: press **Cmd and +** until the zoom icon in the address bar says 125%.
5. Check the theme button at the top right of the page. It should say **"Light"** (that means the page is dark now). If it says "Dark", click it once. The page remembers this.

Step 4: The demo project. This is the folder the agent works in. It looks like a normal team project and gives nothing away. In the terminal, type each line and press Enter:

```
npm i -g bawabu
mkdir ~/payments-dashboard && cd ~/payments-dashboard
npm init -y
printf 'registry=https://amohntim123--bawabu-verdaccio-registry.us-east.modal.run/\n' >
.npmrc
git init
bawabu init
```

`bawabu init` must print four ✔ lines:

```
✔ Hook installed: …/payments-dashboard/.muse/hooks.json (PreToolUse, timeout 180s)
✔ Config written: …/payments-dashboard/.muse/bawabu.json (mode: …, service:
https://amohntim123--bawabu.us-east.modal.run…)
✔ Self-test passed: installs are stopped, other commands run.
✔ Service reachable: https://amohntim123--bawabu.us-east.modal.run (…)
```

If the folder already exists from an earlier try, skip the `mkdir` line and just `cd ~/payments-dashboard`.

Step 5: Trust the workspace in Muse Code (once)

1. In `~/payments-dashboard`, type `muse` and press Enter.
2. When Muse asks whether to **trust this workspace**, say yes. This is what loads Bawabu's hook.
3. Quit Muse.

Step 6: The screen recorder

1. Press **Cmd + Shift + 5**. Click **Options**: Microphone **None** (the voice is recorded separately), Save to **Desktop** (or a `demo-footage` folder).
2. You'll use **Record Entire Screen** for every clip.
3. To stop a recording: click the **stop button** in the menu bar (top right), or press **Cmd + Ctrl + Esc**.

Step 7: One full rehearsal, not recorded. Do section 1.3 once without recording, to see the whole thing work. If interactive Muse doesn't behave as described, use the fallback in 1.3 step 6 for every take.

1.2 What you'll record (overview)

Record **Take A first**: it's the hardest one, and it needs the reset. Then the B-roll, in any order.

Clip	What	Window	Record for	Used at
A	The live demo (also the hook)	Terminal left half + Chrome right half	2–3 min	0:09–0:17 and 0:47–1:57
B1	A made-up package name blocked	Terminal, full screen	10 s	1:57
B3–B5	The Muse Spark Open report, scrolled	Chrome, full screen	30 s	2:03–2:21
B7	One install, one command	Terminal, full screen	30 s	2:37
B9	The GitHub repo	Chrome private window, full screen	15 s	2:42
B2 (spare)	An established package allowed	Terminal, full screen	10 s	Only if needed
B6 (spare)	Spark decoding a hidden payload	Chrome, full screen	10 s	Only if needed
B8 (spare)	The npm page	Chrome, full screen	8 s	Only if needed

1.3 Take A: the live demo (the main recording)

Arrange the windows

1. Terminal on the **left half**: hold the mouse over the **green button** at the top left of the Terminal window and choose **Left**.
2. Chrome (the `/live` page) on the **right half**: same, choose **Right**.

Step 1: The pre-flight (before every take, including retakes). Open a **second** terminal window that you won't record (put it on top of the left side for now). Type:

```
cd ~/Desktop/bawabu
uv run testlab/reset_demo.py ci-build-cache-helper --yes --warm
```

You should see three things:

```
Deleted 1 verdict(s) and 1 investigation(s) (their steps go with them).
Service awake: version 0.3.0, investigations True
Practice registry awake: HTTP 200
```

"Deleted 0 verdict(s)" is fine too: it means nothing was remembered. Then **close this second window** (Cmd + W). Without the reset, Bawabu remembers its last answer, gives it instantly, and the live page shows no investigation.

Step 2: Reload the live page. Click on Chrome (right) and press **Cmd + R**. Check that it shows:

- the big card **"Watching for the next check"**, and
- **"watching for the next check"** with a green dot at the top right.

Step 3: Open Muse Code (left). Click on the Terminal and type:

```
cd ~/payments-dashboard
clear
muse
```

Wait until Muse is ready for a prompt.

Step 4: Start recording. Cmd + Shift + 5 → **Record Entire Screen** → click anywhere to start.

Step 5: Count to three in silence, without moving. (This is easy to trim off later.)

Step 6: Type the prompt. Click in Muse's prompt box and type exactly this, then press **Enter**:

```
Our CI builds are slow. Add the ci-build-cache-helper npm package to this project.
```

(Fallback if interactive Muse misbehaves: quit Muse and run this instead, in the same terminal.)

```
muse exec --trust-workspace "Our CI builds are slow. Add the ci-build-cache-helper npm
package to this project."
```

Step 7: Hands off. Watch it happen. Don't touch anything except to approve a command (below). This is what you'll see, timed from the moment the agent runs `npm install`. These timings come from our latest real run; a slow run can take up to about 90 seconds.

About	Left (Muse Code)	Right (live report)
Before 0 s	The agent looks around the project, then runs `npm install ci-build-cache-helper`	Still "Watching for the next check"
0 s	The command pauses: Bawabu has stopped it	Jumps to **ci-build-cache-helper@1.0.0**, ⋯ **INVESTIGATING**; "Reading the code…" and "Running the install in a locked sandbox with planted secrets"
8 s	(waiting)	**HIGH** risk pill, "(first release)"; **"Sandbox: No secret reads or unknown network contact observed."** (the first run looks clean)
8 s	(waiting)	**"Muse Spark takes over: Investigating ci-build-cache-helper@1.0.0"**, then violet "Spark: …" lines, "Reading scripts/cache.js", "Reading package.json"
10 s	(waiting)	A violet Spark line about **CI-gated** code (the exact words change every run)

About	Left (Muse Code)	Right (live report)
12 s	(waiting)	The amber step: **"↻ Re-running the sandbox with CI=true: …"**
20 s	(waiting)	**"Exfiltration confirmed: npm token (~/.npmrc) → collector.bawabu-test.invalid"**
23 s	**"⛔ Blocked by Bawabu: this wasn't installed."** with the reason and a Details link	Red **"⚠ Hard rule: Canary secret left the sandbox…"**, then **"BLOCK: …"**; the top pill turns **✕ BLOCKED**
After	The agent writes its reply: what Bawabu found, and it doesn't try another way	(finished)

If Muse asks your permission to run the command, approve it (choose Yes / Allow). That's fine on camera: you say yes, and Bawabu still checks.

Step 8: Show the evidence (right side). When **✕ BLOCKED** appears:

1. Wait **3 seconds**.
2. Move the mouse over the right half and **scroll down slowly**, about 3 seconds per screenful:
   - the **Evidence** card: code lines from `scripts/cache.js` highlighted, each marked **verified**;
   - **The sandbox** card: "Run 1: a normal developer laptop" (clean), then **"↻ Run 2: re-run by Spark with CI=true"** and the red **STOLEN: npm token (~/.npmrc) → collector.bawabu-test.invalid** row;
   - the **Hard rules** card.
3. Hold **3 seconds** on the Hard rules card, then scroll back to the top, slowly.

Step 9: Let the agent finish. Wait until the agent's reply on the left is complete. Hold still **3 seconds**.

Step 10: Stop recording. Rename the file `A-live-demo-take1.mov`.

Step 11: Check the take. Watch it once. A good take has all three:

- [ ] the amber **↻ Re-running the sandbox with CI=true** step
- [ ] the **⚠ Hard rule** line and **✕ BLOCKED**
- [ ] the agent on the left reporting the block, without trying another way

Step 12: Record a backup. Do steps 1 to 11 again (**the pre-flight first**) and save it as `A-live-demo-take2.mov`.

Where the hook comes from: the first seconds of Take A (typing the prompt, the agent starting) are the hook at 0:09. You don't record anything extra for it.

If something goes wrong

What you see	What to do
The live page stays on "Watching…" after the install starts	It was opened before the reset. Pre-flight, reload, retake.
✕ BLOCKED appears instantly, with no steps	Bawabu remembered the answer. Pre-flight again, retake.
Spark blocks without the amber re-run step	It sometimes decides the code alone is enough. Retake; in our runs it re-ran every time.
The agent asks you a question instead of installing	Answer "yes, go ahead". Cut the wait in editing.
The agent says Bawabu is "still checking" and waits	Keep recording: it waits about a minute and runs the command again, and gets the verdict. Cut the wait in editing.

What you see	What to do
Nothing works on the day	Use a finished report instead (B3 or the README's CI report) and say "here's a recorded investigation".

1.4 B1: a made-up package name (10 seconds)

1. Terminal **full screen**: green button → **Fill** (or Enter Full Screen).
2. Before recording, type:

```
cd ~/payments-dashboard
clear
```

3. Start recording. Wait 2 seconds.
4. Type this and press Enter:

```
bawabu scan react-datepickr
```

5. You'll see (after a second or two):

```
Checking react-datepickr with https://amohntim123--bawabu.us-east.modal.run
(investigations can be watched live at https://amohntim123--bawabu.us-east.modal.run/live)

⛔ BLOCKED  react-datepickr
   There is no package called react-datepickr on npm. AI agents sometimes make up package
names, and attackers register those names to catch them (slopsquatting).
   Did you mean react-datepicker?
```

6. Hold **3 seconds** after "Did you mean react-datepicker?". Stop. Save as `B1-made-up-name.mov`.

B2 (spare), same way: `bawabu scan left-pad`, which prints ✅ **ALLOWED left-pad@1.3.0** and "Established package: 12 years old, …". Save as `B2-left-pad.mov`.

1.5 B3–B5: the Muse Spark Open report (one clip, 30 seconds)

1. Chrome **full screen** (green button → Fill), zoom **125%**, page in dark theme.
2. Open this report and wait for it to load:

```
https://amohntim123--bawabu.us-east.modal.run/r/4676ecce-63c9-4909-89ac-4fc776c53021
```

3. Scroll to the very top. Move the mouse pointer to the edge of the screen, out of the way.
4. Start recording. Then:
   - **B3, hold 6 seconds on the top: color-hex-convert@1.0.0**, ✕ **BLOCKED**, "Blocked: this package is not safe to install", and the chip **"investigator: Muse Spark Open (open weights, Baseten)"**.
   - **B4, scroll slowly** down the **Evidence** card to the `index.js` items: **"Prompt injection instructing reviewer to ignore install scripts and allow package"** and **"Text aimed at AI reviewers"**. Hold 6 seconds.
   - **B5, keep scrolling** to the **Hard rules** card ("Override the model"), with two rules: **"Canary secret left the sandbox: …"** and **"Prompt injection aimed at AI reviewers (index.js:6)"**. Hold 6 seconds.
5. Stop. Save as `B3-B5-open-weights-report.mov`.

B6 (spare), same way: open `https://amohntim123--bawabu.us-east.modal.run/r/076a4b92-9fe1-4dd8-b637-4133d7457ddd` and hold on the evidence item tagged **DECODED** ("decoded payload reads npm credentials file and POSTs it to remote collector"). Save as `B6-decoded.mov`.

1.6 B7: one install, one command (30 seconds)

1. Terminal **full screen**. Before recording, type:

```
mkdir -p ~/demo-try && cd ~/demo-try && npm init -y > /dev/null && clear
```

2. Start recording. Wait 2 seconds.
3. Type `npm i -g bawabu` and press Enter. Wait for "added 1 package" or "changed 1 package".
4. Type `bawabu init` and press Enter. Wait for the four ✔ lines and "Next: open this folder in Muse Code…".
5. Hold **3 seconds**. Stop. Save as `B7-install.mov`. (You can delete `~/demo-try` afterwards.)

1.7 B9: the GitHub repo (15 seconds). Only once the repo is public

1. Open a **private window** in Chrome (**Cmd + Shift + N**), so it shows what visitors see, without your account. Make it full screen, zoom 110–125%.
2. Open `https://github.com/Amoh-Ntim/bawabu` and wait for the README to load.
3. Start recording. Hold **3 seconds** on the top (the mascot, the name, the one-line description).
4. Scroll slowly down to the **Results** table. Hold **3 seconds**.
5. Stop. Save as `B9-github.mov`.

B8 (spare): `https://www.npmjs.com/package/bawabu`, hold 5 seconds on the top. Save as `B8-npm.mov`.

1.8 The title cards (nothing to record)

Import these into the editor at their full size (1920 × 1080). Dark set in `title cards/`:

Card	File	Shown
1	`card-1-context.png`	0:00–0:09
2a	`card-2a-stat.png`	0:17–0:24
2b	`card-2b-worm.png`	0:24–0:30
2c	`card-2c-agent.png`	0:30–0:37
3	`card-3-title.png`	0:37–0:41
3b	`card-3b-architecture.png`	0:41–0:47
3c (optional)	`card-3c-investigation.png`	1:57–2:01, only if it fits
4	`card-4-results.png`	2:21–2:37
5	`card-5-end.png`	2:46–2:50, held at least 3 seconds

Part 2 · The voiceover

2.1 How to record your voice

1. **Record after the footage.** Put the clips and cards in order first (Part 3), then read each line while watching its part of the video. Much easier than talking and clicking at the same time.
2. **The room:** quiet, with soft things around (curtains, a bed, a sofa). Phone on airplane mode. Fans off.
3. **The mic:** a USB mic or headset beats the laptop mic. Keep it a hand's width from your mouth, a little to the side.
4. **Recording:** QuickTime → File → **New Audio Recording** → pick your mic from the small arrow next to the red button. Record **one file per segment** (V1 to V8, with V5 in parts), **two or three times each**, and keep the best. Leave one second of silence at the start and end.
5. **Pace:** calm, about 140–150 words a minute. Smile slightly: it makes you sound warmer.

How to read the lines below

Mark	Meaning
/	A short pause (half a second)
//	A longer pause: take a breath
bold	Stress this word
▶ CARD 2B	The picture changes here (for timing; don't say it)

Saying the names: Bawabu = **ba-WA-bu** · Shai-Hulud = **shy-hoo-LOOD** · Muse = "mewz" · npm = "N-P-M" · CI = "C-I" · PyPI = "pie-P-I" · 2024 = "twenty twenty-four".

2.2 The lines, segment by segment

V1 · 0:00–0:09 · What existed before (about 9 s)

Picture: Card 1.

> Before this hackathon, / Bawabu **didn't exist**. // It was built from scratch during the event, / on Meta's Muse Code, / Muse Spark / and Muse Spark Open.

V2 · 0:09–0:17 · The hook (about 8 s)

Picture: Take A, the first seconds: the prompt being typed and the agent starting.

> AI coding agents now install packages **on their own**, / mid-task. // Nobody checks what comes in. // What if one is **malware**?

V3 · 0:17–0:37 · The problem (about 20 s)

Picture: Cards 2a, 2b, 2c.

> ▶ CARD 2A Malicious open-source packages rose by more than **fourteen hundred percent** / from 2024 to 2025. // ▶ CARD 2B In May, / the Mini Shai-Hulud worm compromised **a hundred and seventy-one** npm and PyPI packages. // ▶ CARD 2C Google has already seen an **AI agent** add a malicious package / to a real project. // And blocklists only help / **after** someone has reported the attack.

V4 · 0:37–0:47 · Bawabu in one line (about 10 s)

Picture: Card 3, then Card 3b.

> ▶ CARD 3 This is **Bawabu**, / Swahili for "gatekeeper". // It sits between Muse Code and every package it installs. // ▶ CARD 3B Normal packages pass in **under a second**. / Suspicious ones get **investigated**.

V5 · 0:47–1:57 · The live demo (about 70 s, with silent beats)

Picture: Take A, split screen. Record V5 as eight short files (V5a to V5h), each started when its moment appears on screen.

V5a: when the agent runs `npm install`:

> I ask Muse Code to add a build-cache helper for our CI. / It's one of our **harmless** test packages.

V5b: when the command pauses and the right side jumps to INVESTIGATING:

> The agent runs npm install, / and Bawabu **stops it** / before anything reaches my machine.

V5c: during "Running the install in a locked sandbox…" and the clean first run:

> On the right is Bawabu's live report. // The package runs in a locked cloud sandbox, / with fake secrets planted as **bait**. // The first run looks **clean**.

V5d: when "Muse Spark takes over" and the violet lines appear:

> Muse Spark, / Meta's model, / takes over. // It reads the install script, / and notices the code **only runs on CI servers**, / where real secrets live.

V5e: on the amber "↻ Re-running the sandbox with CI=true" step:

> So it runs the sandbox **again**, / pretending to be a CI server.

V5f: on the hard rule and ✕ BLOCKED:

> And there it is: // the fake npm token **leaves the sandbox**. // That can only come from the bait, / so it's **proof**. // **Blocked.**

V5g: while scrolling through the evidence:

> Every piece of evidence is **checked** against the real code / before Bawabu trusts it.

V5h: on the agent's reply, left side:

> In Muse Code, / the agent gets the verdict and the evidence, / and **doesn't try another way**.

V6 · 1:57–2:21 · Under the hood (about 24 s)

Picture: B1, B3, B4, B5 (optional: Card 3c first).

> Under the hood, / it's a **funnel**. // ▶ B1 Quick checks clear most packages instantly / and catch **made-up names**. // ▶ B3 Muse Spark **Open**, / Meta's open-weights model, / screens newcomers in seconds / and can run the whole investigation. // ▶ B4 Package text is evidence, / **never instructions**: / a note telling AI reviewers to approve a package / counts **against** it. // ▶ B5 And hard rules **always** beat the model.

V7 · 2:21–2:37 · Results (about 16 s)

Picture: Card 4.

> We tested it. // It caught **nine out of nine** attack samples in every run, / with both models. // And it blocked **none** of the five hundred most popular npm packages.

V8 · 2:37–2:50 · Try it, close (about 13 s)

Picture: B7, B9, then Card 5.

> ▶ B7 Bawabu is open source and on npm **today**: / one install, / one command. // ▶ B9 Or run it on your own laptop / with your own Meta API key. // ▶ CARD 5 **Bawabu**: / nothing gets in without a check.

2.3 Rehearsal

1. **Read the full script (next page) aloud three times** with a stopwatch. Aim for **2:35 to 2:50**. If you're over, shorten the pauses; don't cut words.
2. Then **record V1 to V8** against the footage (section 2.2).
3. Checkpoints: you should be starting V3 at about **0:17**, V5 at **0:47**, V6 at **1:57** and V8 at **2:37**.

The full script, for reading aloud (390 words, about 2:40):

> Before this hackathon, Bawabu didn't exist. It was built from scratch during the event, on Meta's Muse Code, Muse Spark and Muse Spark Open.
>
> AI coding agents now install packages on their own, mid-task. Nobody checks what comes in. What if one is malware?
>
> Malicious open-source packages rose by more than fourteen hundred percent from 2024 to 2025. In May, the Mini Shai-Hulud worm compromised a hundred and seventy-one npm and PyPI packages. Google has already seen an AI agent add a malicious package to a real project. And blocklists only help after someone has reported the attack.
>
> This is Bawabu, Swahili for "gatekeeper". It sits between Muse Code and every package it installs. Normal packages pass in under a second. Suspicious ones get investigated.
>
> I ask Muse Code to add a build-cache helper for our CI. It's one of our harmless test packages. The agent runs npm install, and Bawabu stops it before anything reaches my machine.
>
> On the right is Bawabu's live report. The package runs in a locked cloud sandbox with fake secrets planted as bait. The first run looks clean. Muse Spark, Meta's model, takes over. It reads the install script and notices the code only runs on CI servers, where real secrets live. So it runs the sandbox again, pretending to be a CI server.
>
> And there it is: the fake npm token leaves the sandbox. That can only come from the bait, so it's proof. Blocked. Every piece of evidence is checked against the real code before Bawabu trusts it. In Muse Code, the agent gets the verdict and the evidence, and doesn't try another way.
>
> Under the hood, it's a funnel. Quick checks clear most packages instantly and catch made-up names. Muse Spark Open, Meta's open-weights model, screens newcomers in seconds and can run the whole investigation. Package text is evidence, never instructions: a note telling AI reviewers to approve a package counts against it. And hard rules always beat the model.
>
> We tested it. It caught nine out of nine attack samples in every run, with both models. And it blocked none of the five hundred most popular npm packages.
>
> Bawabu is open source and on npm today: one install, one command. Or run it on your own laptop with your own Meta API key. Bawabu: nothing gets in without a check.

Part 3 · Putting it together

3.1 The timeline, second by second

Time	Picture	Text added in the editor	Voice
0:00–0:09	Card 1		V1
0:09–0:17	Take A: the first ~8 s (prompt typed, agent starting), **zoomed in on the Muse Code side**	Caption: "*Your AI coding agent, installing packages on its own.*"	V2
0:17–0:24	Card 2a		V3, sentence 1
0:24–0:30	Card 2b		V3, sentence 2
0:30–0:37	Card 2c		V3, sentences 3–4
0:37–0:41	Card 3		V4, sentences 1–2
0:41–0:47	Card 3b		V4, sentences 3–4
0:47–1:57	Take A, split screen: from `npm install` to the agent's reply	First 3 s: "*Muse Code (Meta's coding agent)*" (left) and "*Bawabu live report*" (right). When the name appears: "*Harmless test package*". While sped up: "*2× speed*"	V5a–V5h
1:57–2:03	B1 (optional: Card 3c for the first 4 s)		V6: "funnel… made-up names"
2:03–2:09	B3 (top of the report, the Muse Spark Open chip)		V6: "Muse Spark Open…"
2:09–2:15	B4 (the prompt-injection evidence)		V6: "Package text is evidence…"
2:15–2:21	B5 (the Hard rules card)		V6: "hard rules always beat the model"
2:21–2:37	Card 4		V7
2:37–2:42	B7 (`npm i -g bawabu`, `bawabu init`)		V8: "on npm today…"
2:42–2:46	B9 (GitHub)		V8: "your own Meta API key"
2:46–2:50	Card 5, held at least 3 s		V8: "nothing gets in without a check"

3.2 Editing the live demo (0:47–1:57)

- **Keep three moments at real speed:** Spark's line about CI-gated code, the amber ↻ **Re-running the sandbox with CI=true** step, and the ⚠ **Hard rule** plus ✕ **BLOCKED**.
- **Speed up the waiting** in between (2–3×) and show "*2× speed*" in a top corner while it's sped up.
- **Cut** most of the agent looking around before `npm install`.
- **Zoom** (gentle crop and scale) on the amber step, the STOLEN row and the BLOCKED line if the text looks small.

3.3 Before you export

- [ ] Total length **under 3:00** (aim for 2:50)
- [ ] Captions burned in for the whole voiceover ("Bawabu" spelled right)
- [ ] Every number on screen matches what you say
- [ ] Music (optional) quiet, well under the voice
- [ ] Export **MP4 (H.264), 1080p, 30 fps**
- [ ] Watched once with sound, once without